

API del proyecto

GA7-220501096-AA5-EV04

Camilo Andrés Tibatá Salguero
Aprendiz

DIEGO ARMANDO BARRERA ARAUJO
Instructor

Complejo tecnológico minero agroempresarial

Regional Antioquia
Puerto Berrio

Análisis y desarrollo de software

Ficha

3235874

Introducción

En el desarrollo de aplicaciones web es fundamental comprobar que las diferentes funcionalidades de una API trabajen correctamente y que exista una comunicación adecuada entre el sistema, la base de datos y el usuario.

Para esta actividad se realizó el proceso de testing de la API construida en la evidencia anterior AA5-EV03, utilizando la herramienta Postman.

El objetivo principal de esta evidencia es comprobar el funcionamiento de los diferentes ENDPOINT de la API mediante solicitudes url, verificando las respuestas obtenidas y validando que la información sea procesada y entregada correctamente.

Durante el desarrollo se realizaron pruebas de los ENDPOINT implementados, se revisó la estructura del proyecto, la base de datos creada y el código utilizado para procesar las solicitudes. Finalmente, se documentaron las pruebas realizadas mediante capturas de pantalla y se verificó el funcionamiento de la API mediante Postman.

Desarrollo

OBJETIVO

Realizar el testing de la API desarrollada en la evidencia AA5-EV03 mediante la herramienta Postman, comprobando el funcionamiento de sus ENDPOINT, la comunicación con la base de datos y las respuestas generadas por el sistema.

1. Base de datos

Como primer paso se realizó la creación y configuración de la base de datos utilizada por el proyecto. Esta base de datos permite almacenar la información necesaria para que la API pueda realizar operaciones de consulta, registro, actualización o eliminación de datos, dependiendo de los ENDPOINT implementados.

La conexión entre la aplicación y la base de datos permite que las solicitudes realizadas mediante la API puedan procesar información almacenada de manera persistente.

```
ULL,correo VARCHAR(120),telefono VARCHAR(40));  
NULL,precio DECIMAL(12,2) NOT NULL,stock INT NOT NULL);  
producto_id BIGINT NOT NULL,cantidad INT NOT NULL,total DECIMAL(12,2) NOT NULL,fecha TIMESTAMP DEFAULT
```

```
CREATE TABLE clientes (id BIGINT AUTO_INCREMENT PRIMARY KEY,nombre VARCHAR(120) NOT NULL,correo VARCHAR(120) NOT NULL);  
CREATE TABLE productos (id BIGINT AUTO_INCREMENT PRIMARY KEY,nombre VARCHAR(120) NOT NULL,precio DECIMAL(12,2) NOT NULL,stock INT NOT NULL);  
CREATE TABLE ventas (id BIGINT AUTO_INCREMENT PRIMARY KEY,cliente_id BIGINT NOT NULL,producto_id BIGINT NOT NULL,cantidad INT NOT NULL,total DECIMAL(12,2) NOT NULL,fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

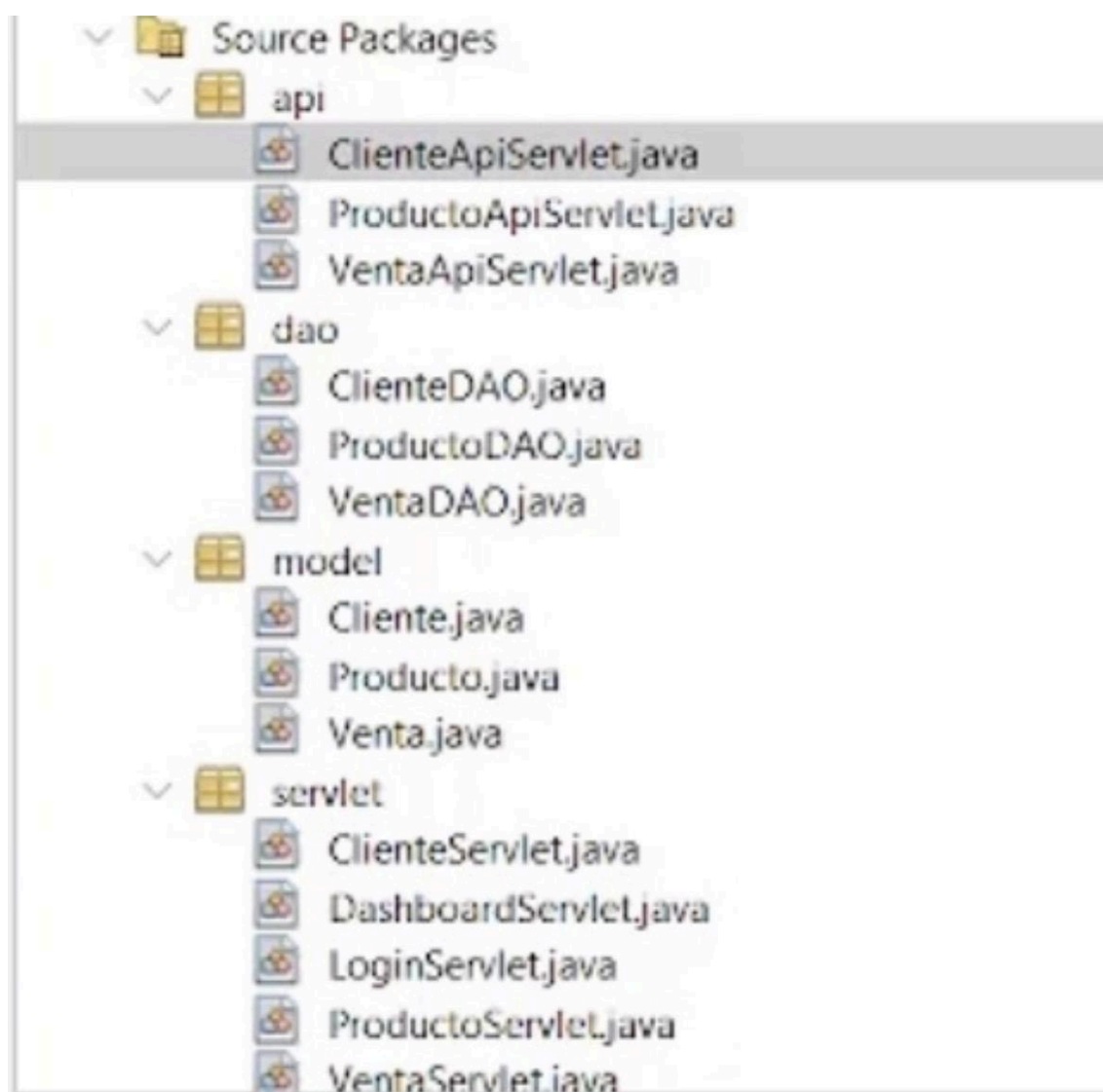
Base de datos creada.

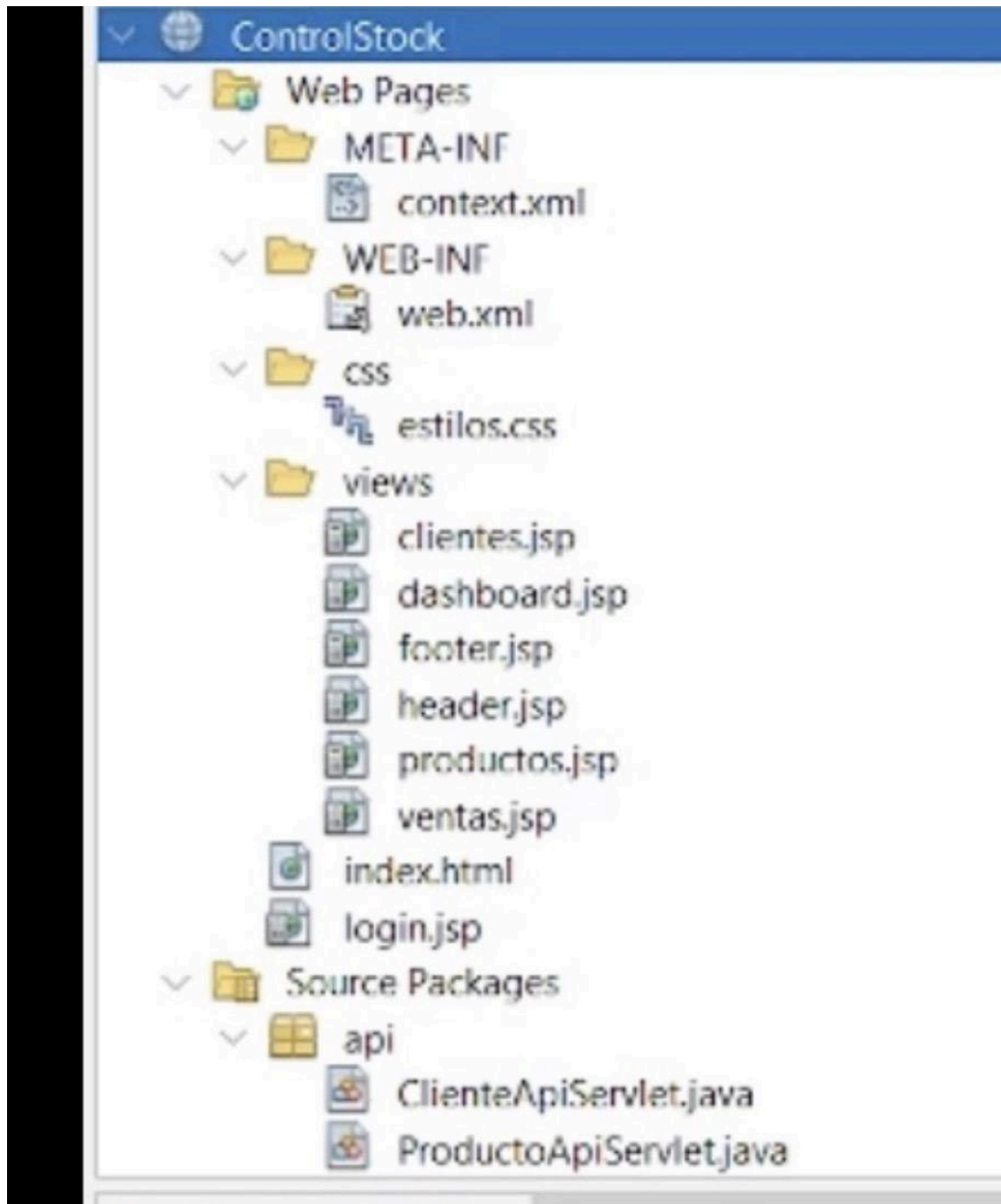
En este pantallazo se evidencia la base de datos utilizada en el proyecto y su estructura correspondiente.

2. Estructura del proyecto

El proyecto fue organizado de manera que cada componente tuviera una función específica. La estructura permite separar la lógica de la aplicación, la conexión con la base de datos, los recursos necesarios y los archivos encargados de recibir y procesar las solicitudes realizadas a la API.

Esta organización facilita el mantenimiento del proyecto y permite identificar con mayor facilidad cada una de las partes que intervienen en el funcionamiento de la API.

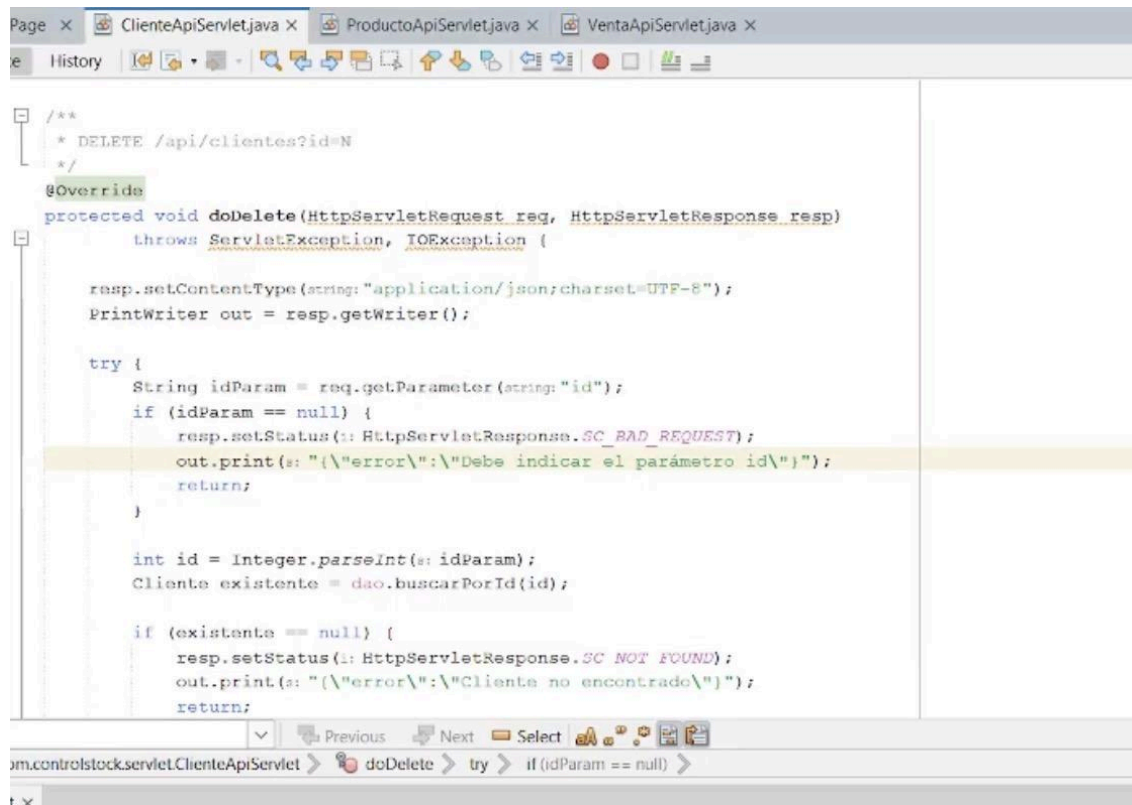




En esta captura se evidencia la organización de las carpetas y archivos que conforman el proyecto.

3. Explicación del código

El código desarrollado contiene la lógica necesaria para que la API pueda recibir solicitudes, procesar la información y generar una respuesta para el cliente. Para esto se utilizan diferentes archivos y clases que permiten organizar las funciones del sistema.



```
/**
 * DELETE /api/clientes?id=N
 */
@Override
protected void doDelete(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {

    resp.setContentType(string: "application/json;charset=UTF-8");
    PrintWriter out = resp.getWriter();

    try {
        String idParam = req.getParameter(string: "id");
        if (idParam == null) {
            resp.setStatus(1: HttpServletResponse.SC_BAD_REQUEST);
            out.print(s: "{ \"error\": \"Debe indicar el parámetro id\" }");
            return;
        }

        int id = Integer.parseInt(s: idParam);
        Cliente existente = dao.buscarPorId(id);

        if (existente == null) {
            resp.setStatus(1: HttpServletResponse.SC_NOT_FOUND);
            out.print(s: "{ \"error\": \"Cliente no encontrade\" }");
            return;
        }
    }
}
```

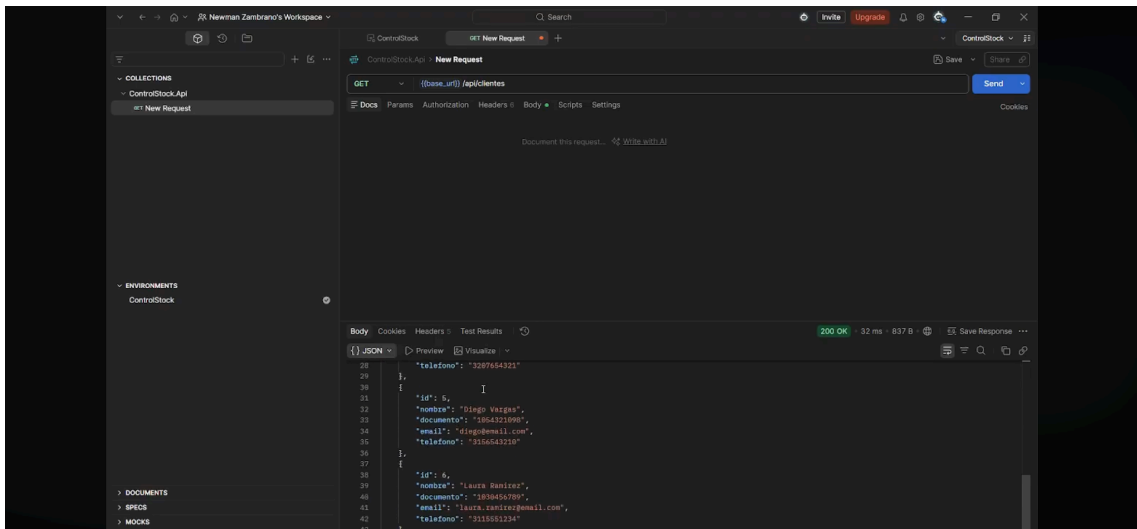
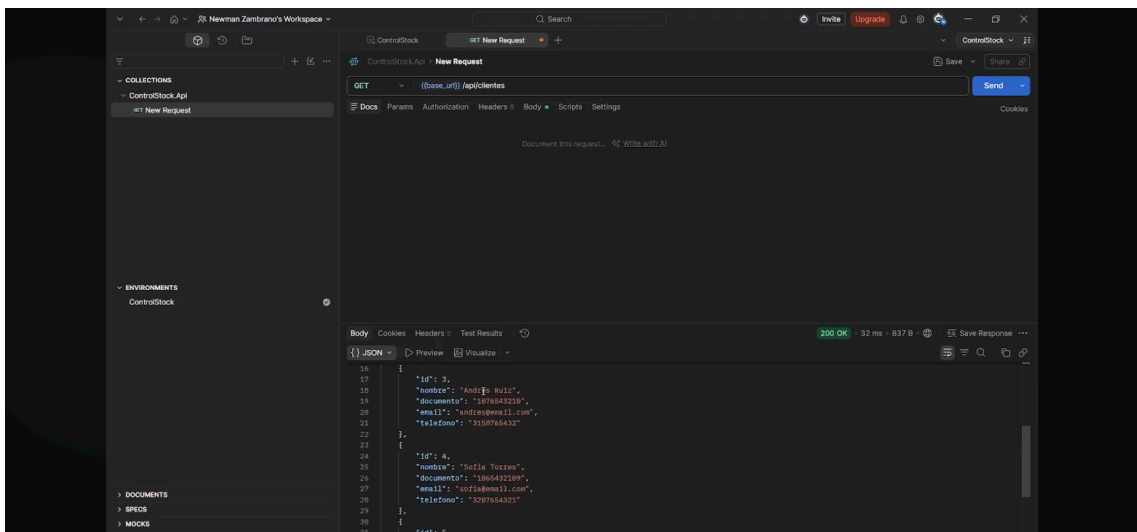
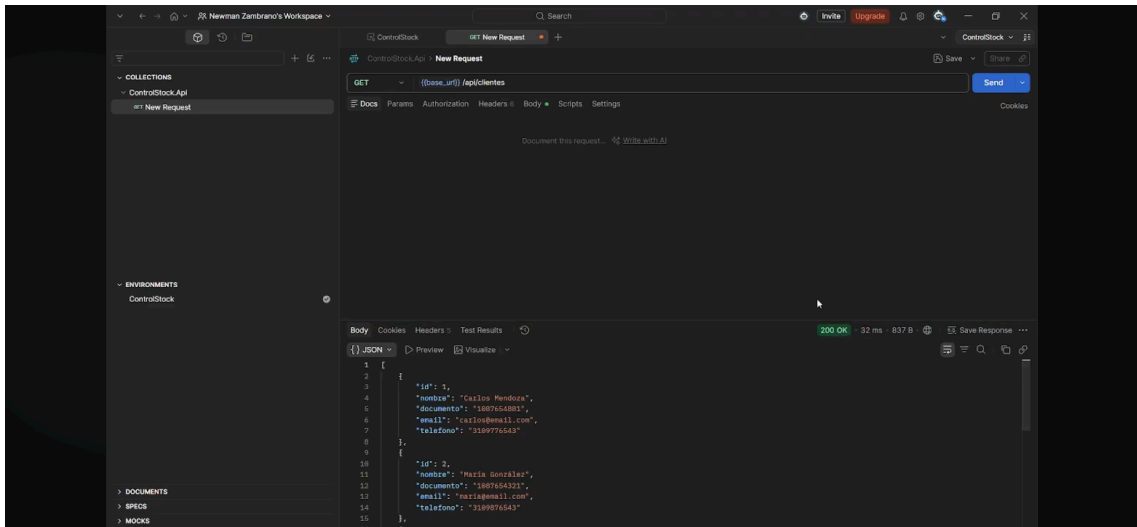
Código del proyecto.

En esta captura se muestra una de las partes principales del código utilizada para el funcionamiento de la API, Código relacionado con la conexión o procesamiento de datos, En este pantallazo se evidencia el código encargado de establecer la comunicación necesaria para trabajar con los datos de la aplicación, Código de la lógica de la API, Esta captura muestra la lógica utilizada para procesar las solicitudes realizadas a los diferentes ENDPOINT, Código de respuesta de la API, En esta parte se evidencia la generación y entrega de la respuesta producida por la API después de procesar una solicitud, Código correspondiente a otro componente de la API, En este pantallazo se muestra otra sección del código que participa en el funcionamiento general del proyecto.

Pruebas de la API utilizando Postman

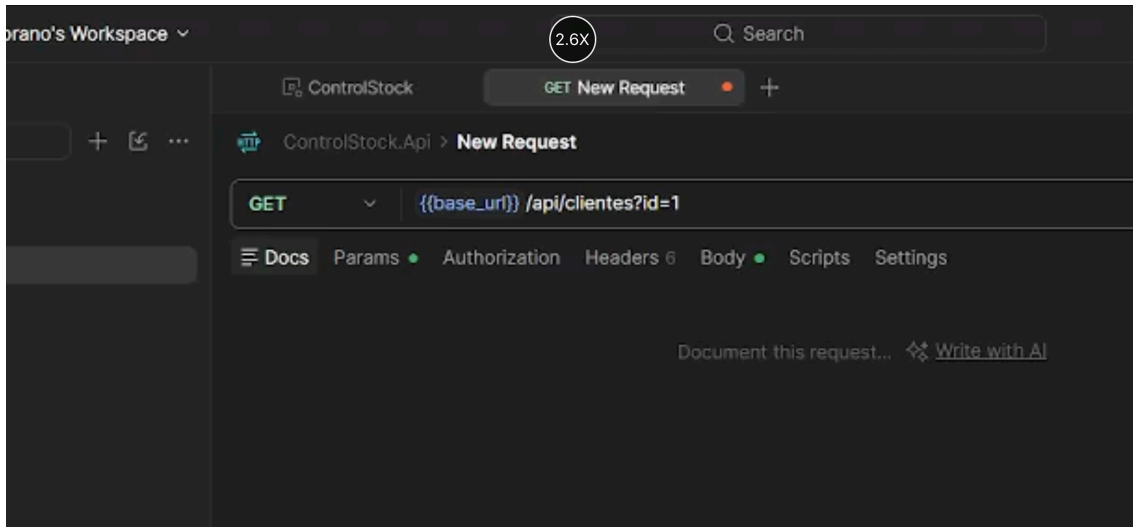
Consultar la lista de productos (GET)

Se realiza una petición HTTP de tipo GET a la ruta `{{base_url}}/api/productos` para obtener el listado completo de los productos registrados en el sistema. La respuesta devuelve un código de estado 200 OK junto con el arreglo en formato JSON que detalla los productos existentes (como la laptop y los accesorios con sus respectivos precios, stock y estado).



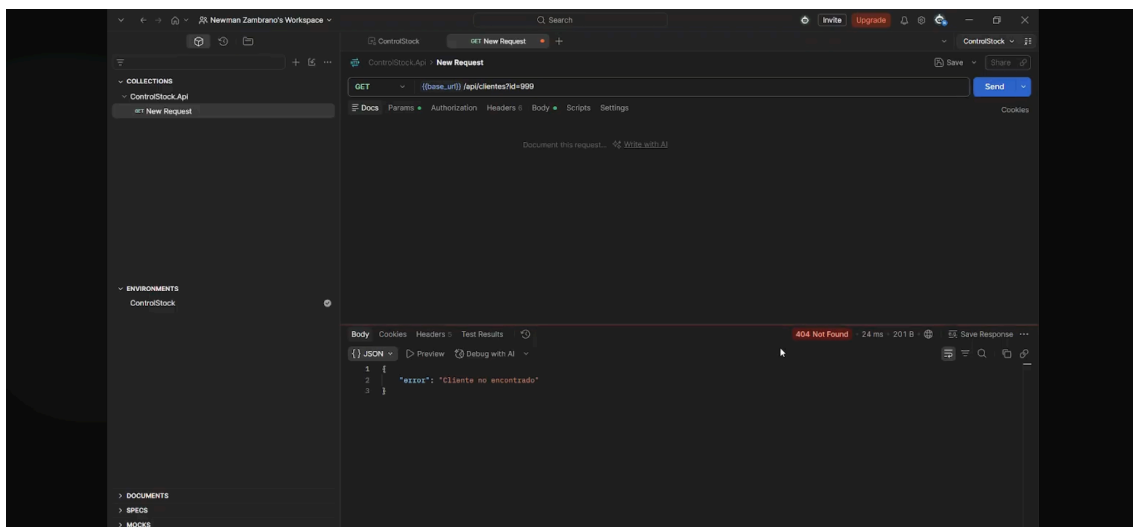
Registrar una nueva venta (POST)

Se ejecuta una petición de tipo POST en la ruta `{{base_url}}/api/ventas` enviando los datos correspondientes en el cuerpo (body) de la petición (id del cliente, id del producto, cantidad, total y estado). La API procesa el registro de forma exitosa y responde con un código 201 Created y un mensaje confirmando que la venta fue registrada correctamente.



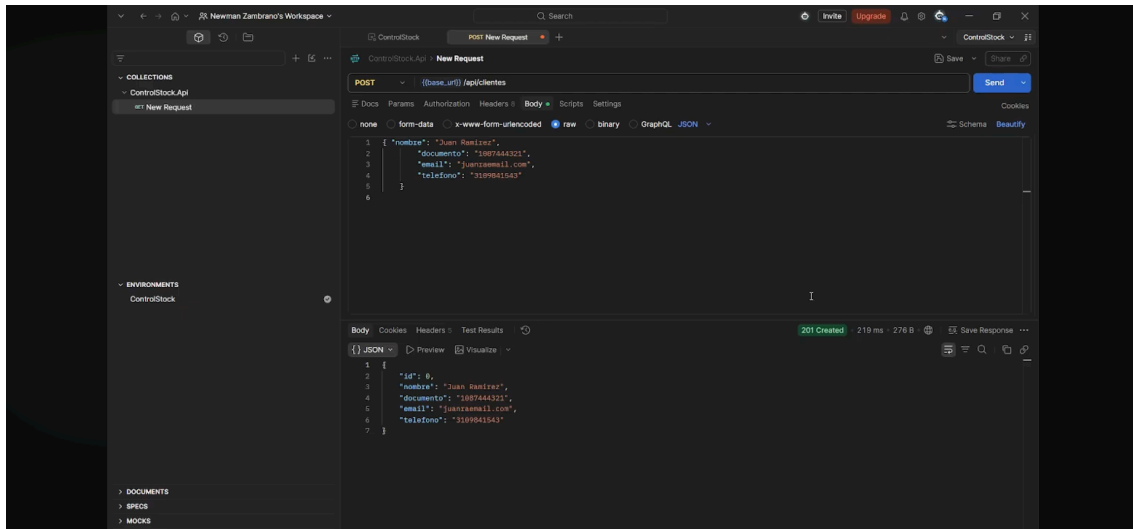
Consultar las ventas registradas (GET)

Se utiliza el método GET apuntando a `{{base_url}}/api/ventas` para consultar todas las transacciones de ventas almacenadas en la base de datos. La respuesta muestra un código 200 OK con el detalle de las compras realizadas por los clientes, incluyendo los productos, cantidades y las fechas exactas de cada movimiento.



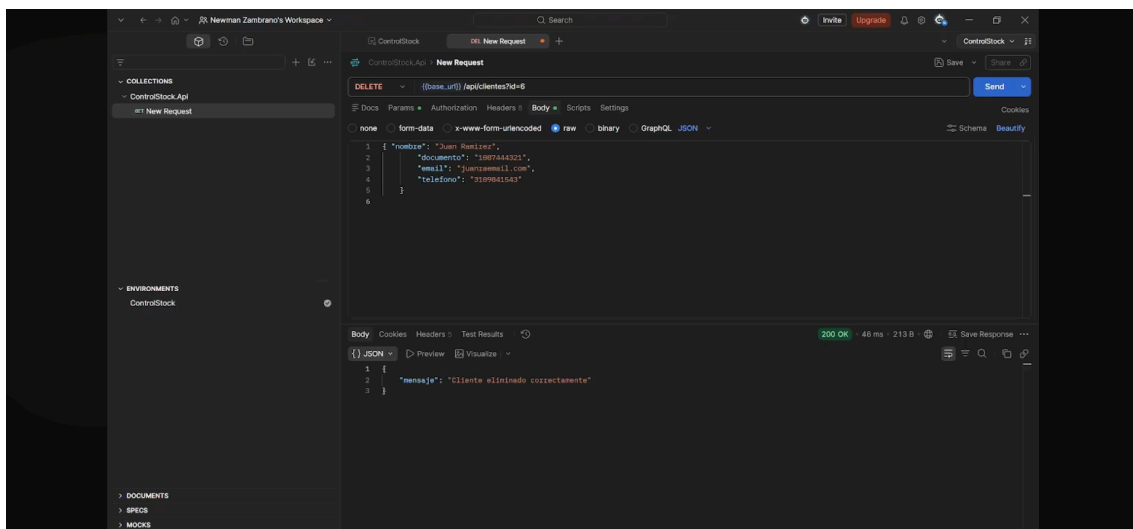
Registrar un nuevo producto (POST)

Mediante una petición de tipo POST hacia `{{base_url}}/api/productos`, se envían los datos de un nuevo artículo (nombre, categoría, stock, precio y estado). El servidor procesa la solicitud y devuelve el código 201 Created, mostrando en la respuesta el objeto creado con su respectivo identificador asignado.



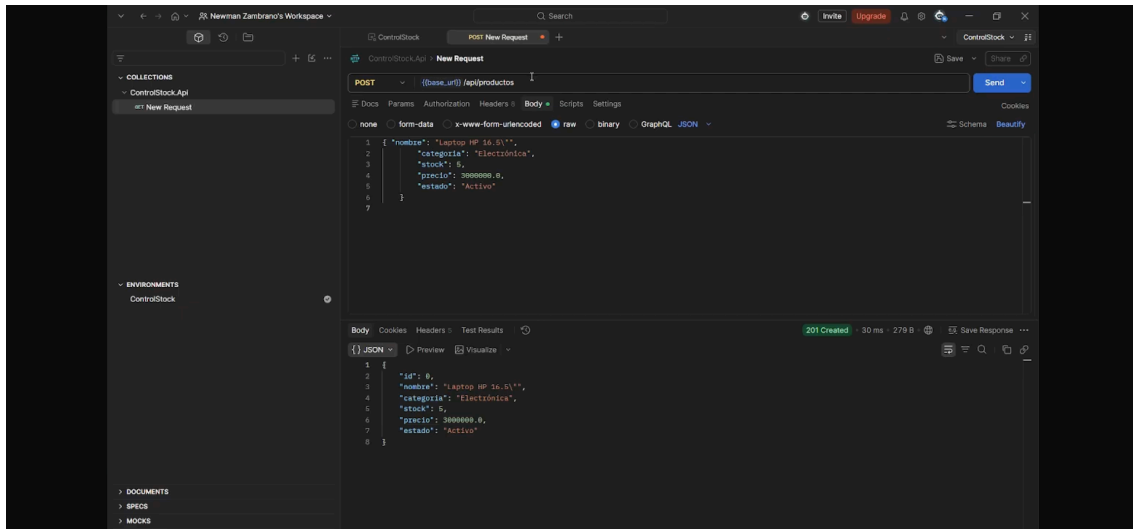
Eliminar un cliente (DELETE)

Se realiza una petición de tipo DELETE apuntando a la ruta `{{base_url}}/api/clientes?id=6` para dar de baja o eliminar a un cliente específico del sistema. La respuesta confirma la acción exitosa con un código 200 OK y un mensaje indicando que el cliente fue eliminado correctamente.



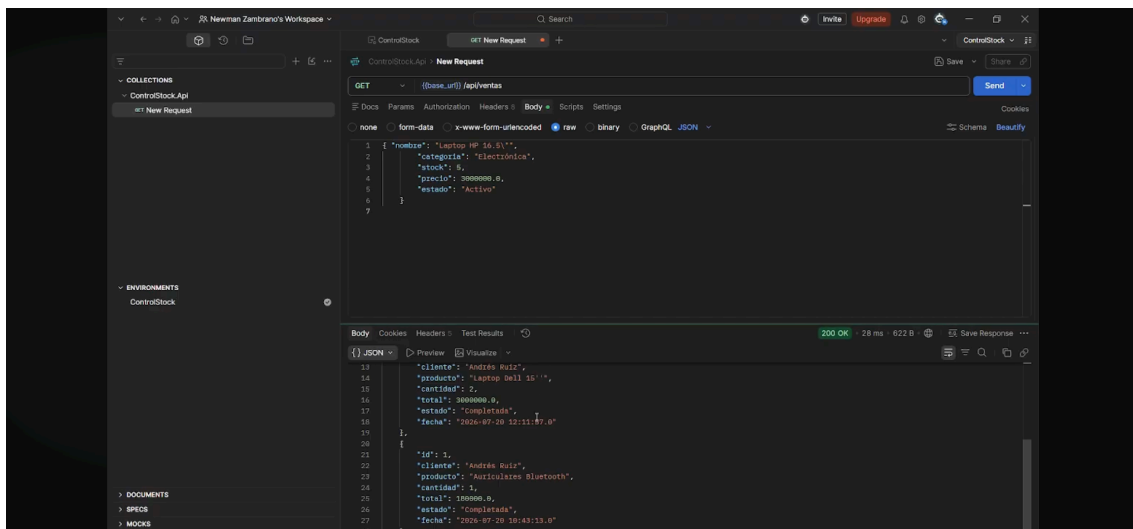
Registrar un nuevo producto (POST)

Se realiza una petición de tipo POST en la ruta `{{base_url}}/api/productos` enviando en el cuerpo (body) los datos en formato JSON de un nuevo artículo (nombre "Laptop HP 16", categoría "Electrónica", stock, precio y estado). La API procesa la solicitud de forma exitosa y responde con un código 201 Created, devolviendo el objeto creado con su respectivo identificador asignado.



Consultar el historial de ventas (GET)

Se ejecuta una petición de tipo GET hacia la ruta `{{base_url}}/api/ventas` con un cuerpo configurado que muestra detalles de productos, obteniendo como resultado un código 200 OK y el listado completo de las transacciones registradas en el sistema (detallando el cliente, producto, cantidad, total, estado y fecha de cada compra).



Resultado general de las pruebas

Las pruebas realizadas mediante Postman permitieron verificar el funcionamiento de los ENDPOINT desarrollados. Se comprobó la comunicación entre el cliente y el servidor y se observaron las respuestas generadas por la API.

El uso de Postman facilitó la identificación de posibles errores y permitió comprobar de manera independiente cada una de las operaciones implementadas, antes de que estas sean utilizadas directamente desde el frontend de la aplicación.

Conclusión

- El desarrollo y la implementación de la API para el proyecto software permitieron consolidar los conocimientos teóricos y prácticos en la construcción de servicios backend robustos, seguros y escalables. Mediante el uso de herramientas de desarrollo, entornos virtuales y sistemas de gestión de bases de datos, se logró estructurar adecuadamente la lógica de negocio y la persistencia de los datos.
- Asimismo, la integración de pruebas exhaustivas a través de Postman facilitó la verificación de los diferentes endpoints, comprobando la correcta recepción de solicitudes, el envío de parámetros y la validación de los códigos de respuesta HTTP (como los estados 200 y de error), garantizando así la interoperabilidad entre el cliente y el servidor.
- En definitiva, esta actividad cumplió a cabalidad con los objetivos planteados al entregar un servicio funcional, documentado y listo para enlazarse con la interfaz de usuario, representando un pilar fundamental en la fase de ejecución del proyecto formativo.
- El uso de Postman como herramienta principal de pruebas facilitó la validación y depuración de cada uno de los endpoints desarrollados en la API. A través de la simulación de peticiones HTTP, fue posible comprobar la correcta transmisión de datos, el cumplimiento de la lógica de negocio y la gestión adecuada de las respuestas del servidor, asegurando así la estabilidad y el funcionamiento óptimo del servicio antes de su integración con el cliente